TaxHub on Azure

Production deployment options — two reference architectures

Open-source & portable · vs · Microsoft-managed

Prepared for JTC Group

The baseline app (both options share this)

Both architectures run the same TaxHub application — only the managed services underneath differ.

- FastHTML 3-pane agentic app, containerised (Docker) — runs unchanged.
- LangGraph orchestrator routing to specialist agents (form-finder, tax-law, entity, AEOI/W-8).
- LLM access through an OpenAI-compatible layer — the provider is a config switch, not a code change.
- Pluggable storage interface (graph or relational) + Blob for form PDFs.
- Per-team isolation, RBAC, invites, chat logging, 👍/👎 feedback and LLM-judge evals already built in.

Option 1 — Open-source & portable RECOMMENDED

Managed Azure PaaS for the plumbing, open-source for the brain — no lock-in, fully pluggable LLMs with Anthropic Claude as default.

Azure Container Apps	Runs the FastHTML app (same Docker image)
Azure Blob Storage	Form PDFs & scraped source documents
Azure Database for PostgreSQL	Entities, obligations, chat logs, feedback & evals · pgvector for embeddings · optional Apache AGE for the citation graph
Azure AI Foundry (model catalog)	Pluggable LLMs — Anthropic Claude default (Opus 4.8 reasoning / Sonnet 4.6 cost); swap to OpenAI, Llama, Mistral with no code change
LangGraph	Agent orchestrator — already in the codebase
Langfuse	Open-source LLM observability: traces, evals, cost, user feedback (self-host or Langfuse Cloud)
Key Vault + Entra ID	Secrets & single sign-on

Option 1 — how it flows

- User → Container Apps (FastHTML) over HTTPS with Entra SSO.
- LangGraph routes each request to specialist agents; every step is traced to Langfuse.
- LLM calls hit Azure AI Foundry via the OpenAI-compatible endpoint — Anthropic Claude by default, swappable per-agent.
- Retrieval runs over PostgreSQL (pgvector hybrid) + PDFs in Blob.
- 👍/👎 feedback and LLM-judge scores flow to Langfuse and the in-app analytics for a full quality loop.

Option 1 — pros & cons

Pros

- ✓ No lock-in — portable across Azure, other clouds, or on-prem.
- ✓ Truly pluggable LLMs: Anthropic default, swap freely; no single-vendor model risk.
- ✓ Reuses the existing codebase (LangGraph, pluggable storage) — minimal rework.
- ✓ Open-source observability you own (Langfuse) — full trace/eval/cost data.
- ✓ Transparent usage-based cost; scale each component independently.

Cons

- ✗ More moving parts to operate (Container Apps, Postgres, Langfuse).
- ✗ You own upgrades, scaling & backups — eased by managed PaaS.
- ✗ Graph queries need pgvector/Apache AGE, or a managed Neo4j AuraDB add-on.
- ✗ Langfuse is another service to run (or pay for Langfuse Cloud).

Option 2 — Microsoft-managed MANAGED

Foundry Agent Service + Microsoft Fabric — least ops, deepest Microsoft integration, at the cost of portability.

Azure AI Foundry Agent Service	Managed agents, built-in tool-calling, threads & multi-agent orchestration (replaces custom LangGraph)
Azure AI Search	Managed vector + hybrid RAG over the corpus
Microsoft Fabric (OneLake)	Unified data lake for tax docs · Lakehouse / Warehouse · Data Factory ingestion pipelines
Power BI	Firmwide compliance & filing dashboards on Fabric data
Azure OpenAI (GPT)	Default models; Anthropic Claude also available in the Foundry catalog
Azure Monitor + Foundry evaluations	Built-in tracing & evaluation
Microsoft Purview	Governance, data lineage & DLP

Option 2 — pros & cons

Pros

- ✓ Fully managed — least ops; Microsoft runs the agents, RAG and scaling.
- ✓ Deep Microsoft integration: Entra, Purview governance, Fabric, Power BI.
- ✓ Enterprise SLA & support — strong fit for a Microsoft-shop like JTC.
- ✓ Built-in agent orchestration, managed RAG and evaluations out of the box.
- ✓ Fabric unifies data + BI for firmwide reporting.

Cons

- ✕ Vendor lock-in — Azure/Foundry/Fabric-specific; low portability.
- ✕ Rework: migrate off LangGraph/Neo4j to Foundry Agents + Fabric.
- ✕ Model choice steered to OpenAI; less control over orchestration internals.
- ✕ Fabric capacity (F-SKUs) + Search + Foundry can be costly & hard to predict.
- ✕ Managed-agent surface is newer and still maturing.

Side by side

Dimension	Option 1 · Open-source	Option 2 · MS-managed
Compute	Container Apps (Docker)	Foundry Agent Service (managed)
Orchestration	LangGraph (code)	Foundry Agents (managed)
LLMs	Foundry catalog — Anthropic default, fully swappable	Azure OpenAI default; Claude via catalog
Data	PostgreSQL (+pgvector/AGE) + Blob	Fabric OneLake + AI Search
Observability	Langfuse (open-source)	Azure Monitor + Foundry evals
Analytics / BI	In-app + Langfuse	Power BI on Fabric
Lock-in	Low — portable	High — Azure-native
Ops burden	Higher	Lower
Rework	Minimal (reuses code)	Significant
Best for	Portability, model choice, cost control	MS-shop, managed ops, firmwide BI

Recommendation

- Default to Option 1 (open-source & portable): keeps Anthropic Claude as the primary model with freedom to swap, reuses the current LangGraph app, and avoids lock-in. Langfuse adds production-grade observability on top of the in-app analytics.
- Choose Option 2 when JTC wants a fully-managed, Microsoft-native stack with Fabric / Power BI for firmwide reporting and accepts Azure lock-in plus the migration effort.
- Hybrid path: start on Option 1 (fast, low-risk, portable), then adopt Fabric / Power BI for analytics later if firmwide BI becomes a priority — the pluggable storage and OpenAI-compatible LLM layer make that incremental, not a rebuild.